

M4. Introduction à la programmation orientée objet



Ce module a pour objectif de vous familiariser avec les **concepts de base de la programmation orientée objet dans le langage Python**. Nous allons explorer les avantages de la programmation orientée objet et étudier la création de classes et d'objets. À la fin de ce module, vous serez en mesure de créer des programmes orientés objet en utilisant Python. Prêt à commencer ? Allons-y !



Concepts de base de la POO



Les avantages de la POO



Les éléments clés de la POO: classes et objets



Review

Concepts de base de la POO

La **programmation orientée objet (POO)** est un *paradigme* de programmation qui organise le code autour d'objets qui représentent des entités du monde réel ou des concepts abstraits. La POO est pleinement intégrée au langage Python.

Contrairement aux autres paradigmes de programmation, la POO consiste à décrire les *objets* utilisés par le programme et/ou l'utilisateur ainsi que leurs relations intrinsèques et leurs fonctions.

On décrira, par exemple, l'objet *Client* et l'objet *Bon de commande*, leurs propriétés propres (nom, prénom, e-mail pour le *Client* ; numéro, date de commande pour le *Bon de commande*). L'objet *Client* aura peut-être comme fonction *Passer une commande* qui générera un objet *Bon de commande*.

 Un **paradigme** est une manière de concevoir un problème et les solutions associées. Le paradigme de POO est régi par un ensemble de concepts qui permettent notamment de mettre en œuvre de manière efficace cette philosophie de programmation.

Voici les **concepts de base de la POO** en Python :

- Les classes.
- Les objets.
- Les attributs (de classes ou d'instances).
- Les méthodes.
- L'encapsulation.
- L'héritage.
- Le polymorphisme.

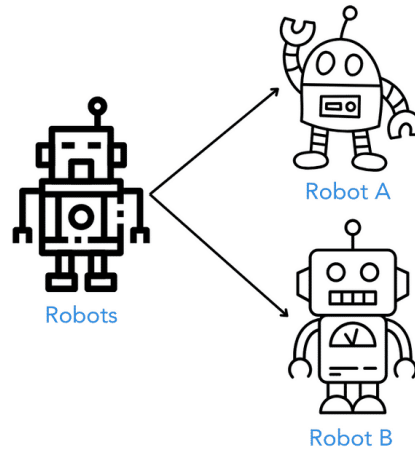
Ce module voulant présenter les concepts et non les détailler, nous allons voir succinctement la place de chaque concept dans la POO.

Les classes

Une classe est une **structure**, un **modèle** qui définit les attributs (variables) et les méthodes (fonctions) communes à un groupe d'objets. Elle agit comme un modèle à partir duquel des objets individuels peuvent être créés. La définition d'une classe utilise le mot-clé **class**.



La classe est le *moule* à partir duquel on crée les objets.

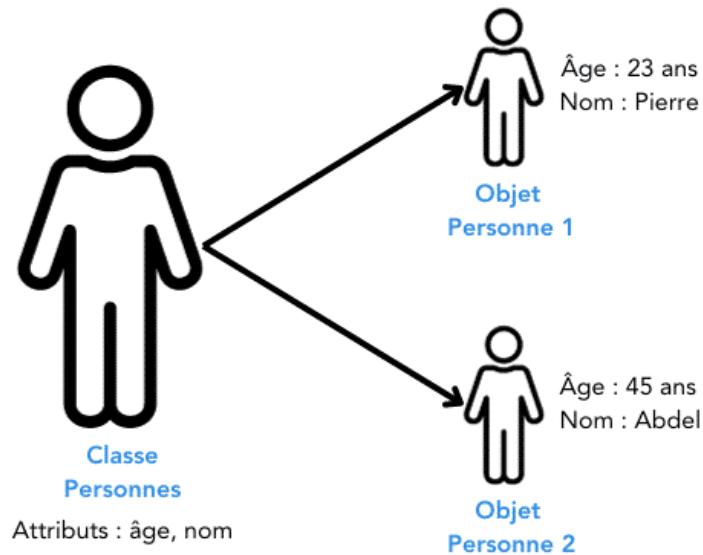


2 objets *Robot A* et *Robot B* sont créés à partir de la classe *Robots*

Les objets et leurs attributs

Un objet est une instance d'une classe. Il est créé à partir de la classe et possède donc les attributs propre à cette classe. Par contre, l'objet possède **ses propres valeurs** d'attributs.

Les **attributs** sont des propriétés héritées de la classe d'objets ou propre à l'objet lui-même.



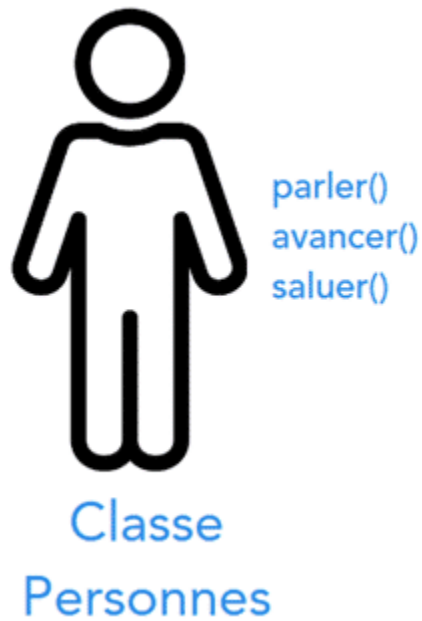
Deux objets Personne 1 et Personne 2 instanciés à partir de la classe Personnes. Les attributs proviennent de la classe mais chaque objet a ses propres valeurs d'attribut.

Les Méthodes

Les méthodes sont des **fonctions définies à l'intérieur d'une classe**. Elles représentent le comportement ou les actions que les objets de la classe peuvent effectuer.

Les méthodes peuvent accéder et modifier les attributs d'un objet.

Par exemple, une classe Personne peut avoir une méthode `parler()` qui affiche un message.



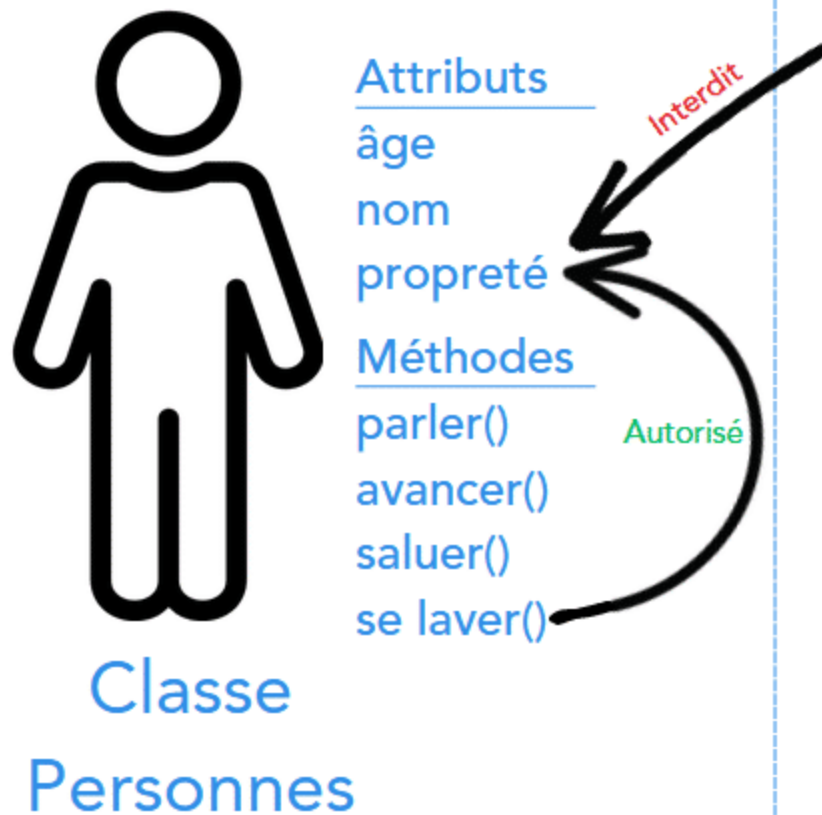
Quelques méthodes de la classe Personnes

L'Encapsulation

L'**encapsulation** est un concept qui consiste à **regrouper les attributs et les méthodes connexes au sein d'une classe pour former une seule entité**.

L'encapsulation permet de cacher les détails internes à une classe et de protéger les données sensibles en définissant des attributs privés et donc inaccessibles depuis l'extérieur de la classe ou en utilisant des méthodes d'accès (getters et setters) pour interagir avec les attributs.

encapsulation



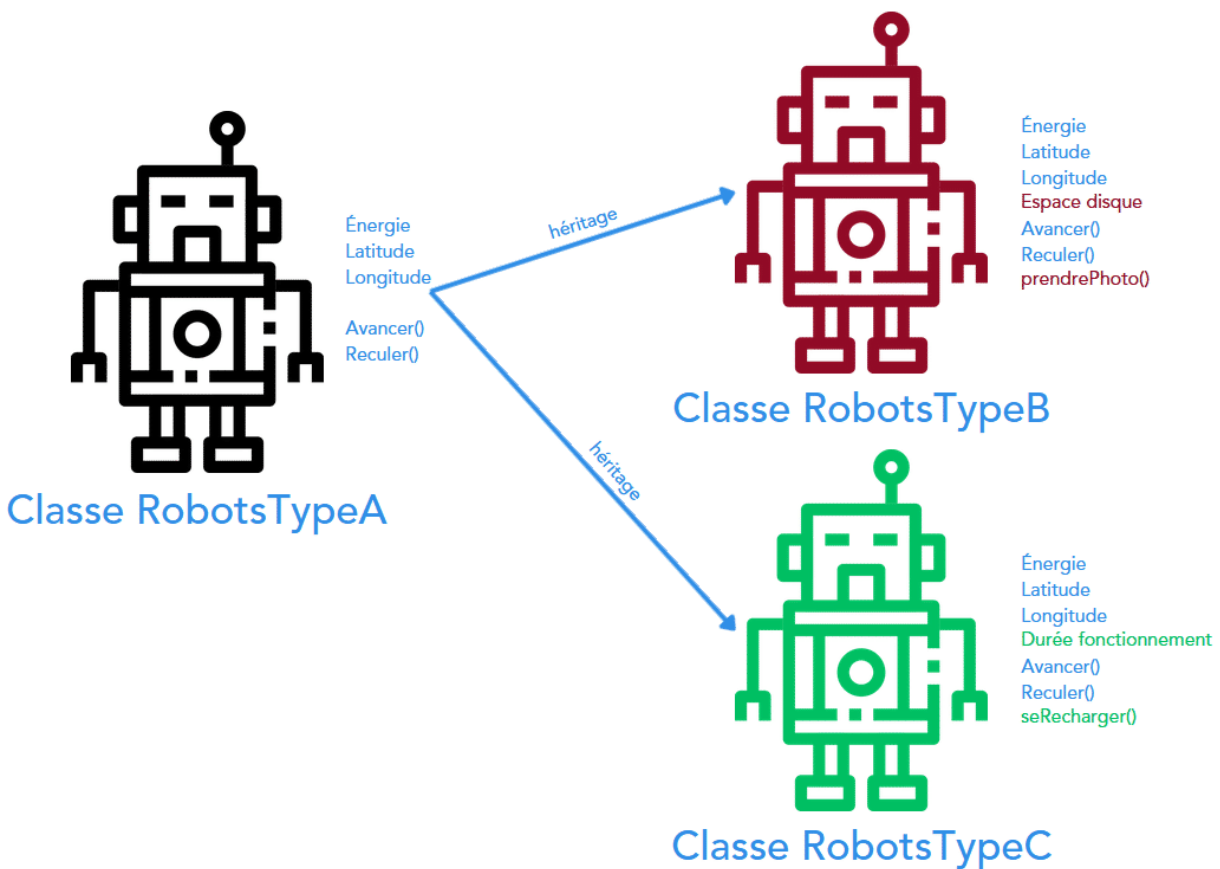
Avec l'encapsulation, l'extérieur d'une classe (ou d'un objet) ne peut modifier les attributs de cette classe (ou de cet objet) ; il faut passer par les méthodes de la classe (ou de l'objet).

L'Héritage

L'héritage permet de créer des **sous-classes** ou **classes enfants** en utilisant les caractéristiques d'une classe existante appelée **superclasse** ou **classe parent**.

Une sous-classe hérite de tous les attributs et de toutes les méthodes de sa superclasse et peut ajouter ou modifier son propre comportement.

L'héritage favorise la réutilisabilité du code et permet de créer une hiérarchie de classes.



Deux sous-classes qui héritent des attributs et des méthodes de la classe parente.

Le Polymorphisme

Le polymorphisme est la capacité d'un objet à **prendre plusieurs formes**.

Le polymorphisme est principalement réalisé grâce à l'héritage et à la redéfinition de méthodes. Il permet d'appeler des méthodes d'une classe de manière générale, sans se soucier du type spécifique de l'objet. Cela facilite l'écriture de code générique et flexible.

CONTINUER

Les avantages de la POO

Au delà d'une apparente difficulté pour les débutants ou d'une impression de surcharge de travail lors de l'écriture d'une application, l'**utilisation de la Programmation Orientée Objet (POO) présente de nombreux avantages.**

Structuration et réutilisation, maintenabilité du code et sécurité

La POO va non seulement permettre d'organiser le code en classes et en objets (ce qui rend le **code plus clair, modulaire et facile à comprendre**) mais également permettre de **maximiser la réutilisation du code existant** (grâce à l'héritage) ; ce qui amène à terme une **économie de temps et d'efforts de développement**.

De plus, en encapsulant les données et les méthodes associées dans des classes, la POO **facilite la maintenance du code** ; les modifications apportées à une classe n'affectant pas les autres parties du programme.

En pratiquant l'encapsulation des données au sein des objets et en contrôlant l'accès à ces données, la POO offre également une **meilleure sécurité** de ces dernières.

Séparation of concern et gestion de la complexité

La POO permet de **séparer les différentes fonctionnalités** ([separation of concern](#)) d'un programme en classes distinctes. Cela permet de diviser un problème complexe en parties plus petites et plus gérables.

En modélisant des concepts du monde réel en classes, la POO permet de **gérer la complexité d'un système en le décomposant en objets interagissant entre eux**.

Lisibilité, évolutivité et intuitivité dans la conception

Au moyen de la POO, il est possible de **mettre en œuvre des solutions logicielles de manière plus naturelle et intuitive** en utilisant des concepts tels que les objets, les relations et les interactions. Cela facilite la modélisation et la résolution des problèmes.

L'application de tous les concepts abordés précédemment rendront indubitablement le code plus lisible et plus compréhensible. Le fait de traiter, manipuler et mettre en place des concepts du monde réel ou des abstractions logiques rendent la compréhension de celui-ci plus aisée.

Cherry on top (la cerise sur le gâteau 🍒) : il est sensiblement **plus aisé de faire évoluer un programme en ajoutant de nouvelles fonctionnalités sans altérer le code existant**. Les modifications sont localisées dans les classes et les méthodes appropriées.

CONTINUER

Les éléments clés de la POO: classes et objets

Lorsque vous voulez regrouper des informations telles que des données sur des employés, il devient intéressant de créer la structure de données correspondante qui contiendra tous les éléments nécessaires permettant de définir un employé selon vos règles métier.

La structure idéale au sein de laquelle nous pouvons définir ces informations **est une classe**.

En pseudo-code, on pourrait définir cette structure de la manière suivante :

```
Employee
  property : name
  property : role
  property : department

  behavior : changeRole(newRole)
  behavior : changeDepartment(newDepartment)
```

Le pseudo-code qui définit une classe Employee, ses propriétés et ses comportements.

La classe est la structure de code de base de la POO qui va permettre de décrire ce que nous entendons dans notre application lorsque nous parlons d'un employé (dans le cas illustré ci-dessus) et doit donc être vue comme **un plan à partir duquel tous nos objets seront créés**.

Dans le cas présent, lorsque nous créerons un **objet employé**, nous créerons en fait une image de notre classe, appelée **instance**, qui représentera dans notre code **un et un seul** employé et pour lequel nous pourrons spécifier toutes les propriétés (nom, rôle, département).

CONTINUER

Review

Remettez ces éléments dans le bon ordre :

≡ Une classe est

une structure, un modèle de base

≡ Un objet est

une instance d'une classe

≡ Un attribut est

partagé par tous les objets de la classe

≡ Une méthode est

une fonction définie à l'intérieur d'une classe

SUBMIT

Sélectionnez les affirmations correctes.

Plusieurs réponses possibles.

☐

La définition d'une classe utilise le mot-clé **class**.

☐

La POO est un frein à l'évolutivité d'un programme

☐

La POO facilite la séparation des différentes fonctionnalités d'un programme

☐

L'encapsulation permet de sécuriser les applications

SUBMIT

Complétez la phrase suivante :

Lorsque nous créons un objet à partir d'une classe, nous créons une image de notre classe. Cette image est appelée une _____ de classe.

Tapez votre réponse ici

SUBMIT